



**Universidad Tecnológica Centroamericana**

**Proyecto Integrador-Equipo**

**Presentado por:**

|                |          |
|----------------|----------|
| Luis Henríquez | 10941366 |
| Danna Casco    | 12241116 |
| Mía Elvir      | 12341202 |

**Presentado a:**

Ing. Kevin Fúnez

**Clase y sección:**

Fundamentos de ciencia de datos    498

**Fecha de entrega:**

Tegucigalpa M.D.C                      24 de marzo de 2026

# Tabla de Contenido

|     |                                           |    |
|-----|-------------------------------------------|----|
| 1.  | INTRODUCCIÓN.....                         | 3  |
| 2.  | OBJETIVOS GENERALES .....                 | 4  |
| 3.  | OBJETIVOS ESPECIFICOS .....               | 4  |
| 4.  | IMPORTACIÓN DE LIBRERÍAS.....             | 5  |
| 5.  | CARGA Y EXPLORACIÓN INICIAL DE DATOS..... | 6  |
| 6.  | LIMPIEZA DE DATOS.....                    | 7  |
| 7.  | EXPLORACIÓN NUMÉRICA.....                 | 8  |
| 8.  | EXPLORACIÓN GRÁFICA.....                  | 10 |
| 9.  | MODELADO .....                            | 12 |
| 10. | EVALUACIÓN DEL MODELO .....               | 15 |
| 11. | CONCLUSIONES .....                        | 19 |
| 12. | BIBLIOGRAFÍA.....                         | 20 |

## **1. INTRODUCCIÓN**

En la actualidad enfocada en un mundo lleno de empresas, dentro de un entorno cada vez más competitivo, estas mismas dependen directamente de sus clientes para garantizar su sostenibilidad, tal como ha ocurrido históricamente. Sin embargo, existen diversos factores y contextos que pueden influir en la permanencia de un cliente dentro de una empresa.

La fuga de clientes actualmente representa un desafío significativo para las organizaciones. Para asegurar el éxito de estas, es fundamental aprovechar la experiencia histórica y los datos disponibles que permitan identificar qué clientes, o bajo qué circunstancias, existe una mayor probabilidad de abandono. Asimismo, comprender los factores que influyen en esta decisión resulta clave para desarrollar estrategias efectivas orientadas a aumentar la retención de clientes.

En el presente proyecto integrador se desarrollará un modelo analítico aplicado a una empresa del sector Telco, con el objetivo de evaluar y predecir la fuga de clientes, identificando patrones, factores clave y probabilidades asociadas al abandono de la empresa.

## **2. OBJETIVOS GENERALES**

Identificar y analizar los factores que influyen en que los clientes abandonen la empresa (fuga), por medio del uso de un proceso analítico y modelos predictivos en Python, con el objetivo de comprender patrones en la historia de fuga de los clientes.

## **3. OBJETIVOS ESPECIFICOS**

Realizar un análisis exploratorio de datos (EDA) para comprender la estructura, distribución y calidad inicial del conjunto de datos a analizar.

Identificar las variables más relevantes que influyen en la fuga de clientes mediante el uso de técnicas estadísticas y el análisis de datos históricos.

Desarrollar y entrenar un modelo predictivo que permita estimar la probabilidad de fuga de los clientes en función de los factores identificados.

Interpretar los resultados obtenidos para identificar patrones de fuga y, a partir de estos, proponer estrategias orientadas a la retención de clientes.

## 4. IMPORTACIÓN DE LIBRERÍAS

Para iniciar el proceso de construcción del modelo de fuga de clientes, se configura el entorno de trabajo mediante la importación de las librerías necesarias. Estas herramientas ya establecidas apoyan el desarrollo del proceso de manera más eficiente y sencilla para el usuario.

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import statsmodels.api as sm
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn import metrics
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report, roc_curve, auc, roc_auc_score, RocCurveDisplay
from google.colab import drive
```

Se importan diversas librerías, como:

| Librería                                                 | Función                                                   |
|----------------------------------------------------------|-----------------------------------------------------------|
| Pandas                                                   | Manipulación de los datos como tablas                     |
| Numpy                                                    | Aplicaciones estadísticas                                 |
| Matplotlib y Seaborn                                     | Visualización por medio de gráficas                       |
| statsmodels.api                                          | Implementación de modelos                                 |
| train_test_split, StandardScaler y LogisticRegression    | Aprendizaje de máquina, incluyendo entrenamiento y prueba |
| accuracy_score, confusion_matrix y classification_report | Evaluación del modelo                                     |

## 5. CARGA Y EXPLORACIÓN INICIAL DE DATOS

Se realiza la carga de los datos al entorno para comenzar el proceso de modelado utilizando la librería pandas para ordenar la información en tablas y de esta manera poder visualizar de primera vista el conjunto de datos.

```
df = pd.read_csv("/content/TelcoClientesFuga.csv")
df.head(4)
```

|   | customerID | gender | SeniorCitizen | Partner | Dependents | tenure | PhoneService | MultipleLines    | InternetService | OnlineSecurity | ... | DeviceProtection | TechSupport | StreamingTV |
|---|------------|--------|---------------|---------|------------|--------|--------------|------------------|-----------------|----------------|-----|------------------|-------------|-------------|
| 0 | 7590-VHVEG | Female | 0             | Yes     | No         | 1      | No           | No phone service | DSL             | No             | ... | No               | No          | No          |
| 1 | 5575-GNVDE | Male   | 0             | No      | No         | 34     | Yes          | No               | DSL             | Yes            | ... | Yes              | No          | No          |
| 2 | 3668-QPYBK | Male   | 0             | No      | No         | 2      | Yes          | No               | DSL             | Yes            | ... | No               | No          | No          |
| 3 | 7795-CFOCW | Male   | 0             | No      | No         | 45     | No           | No phone service | DSL             | Yes            | ... | Yes              | Yes         | No          |

4 rows × 21 columns

A partir de una vista previa de la tabla se visualiza la estructura general del conjunto por medio de la revisión de tipos de datos para revisar existencia de inconsistencias.

```
df.info()

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 7040 entries, 0 to 7039
Data columns (total 21 columns):
#   Column                Non-Null Count  Dtype
---  -
0   customerID            7040 non-null   object
1   gender                 7040 non-null   object
2   SeniorCitizen          7040 non-null   int64
3   Partner                7040 non-null   object
4   Dependents             7040 non-null   object
5   tenure                 7040 non-null   int64
6   PhoneService           7040 non-null   object
7   MultipleLines          7040 non-null   object
8   InternetService        7040 non-null   object
9   OnlineSecurity         7040 non-null   object
10  OnlineBackup           7040 non-null   object
11  DeviceProtection       7040 non-null   object
12  TechSupport            7040 non-null   object
13  StreamingTV            7040 non-null   object
14  StreamingMovies        7040 non-null   object
15  Contract               7040 non-null   object
16  PaperlessBilling       7040 non-null   object
17  PaymentMethod          7040 non-null   object
18  MonthlyCharges         7040 non-null   float64
19  TotalCharges           7040 non-null   object
20  Churn                  7040 non-null   object
dtypes: float64(1), int64(2), object(18)
memory usage: 1.1+ MB
```

## 6. LIMPIEZA DE DATOS

Como paso inicial de la limpieza, se convierte el valor de TotalCharges a valor numérico ya que no se puede realizar el análisis matemático a una variable de tipo Object.

```
df["TotalCharges"] = pd.to_numeric(df["TotalCharges"], errors="coerce")
```

Se revisa si existen valores nulos, de esta forma, son eliminados para evitar inconvenientes al momento del análisis

```
df.isnull().sum()
```

|               |   |
|---------------|---|
|               | 0 |
| customerID    | 0 |
| gender        | 0 |
| SeniorCitizen | 0 |

Al identificar valores nulos en alguna columna, se procede a revisarlos para comprender la razón de su presencia (En este contexto, la columna con valores nulos es TotalCharges).

```
df[df["TotalCharges"].isnull()][["tenure", "MonthlyCharges", "TotalCharges", "Churn"]]
```

|      | tenure | MonthlyCharges | TotalCharges | Churn |
|------|--------|----------------|--------------|-------|
| 1079 | 0      | 25.75          | NaN          | No    |
| 1337 | 0      | 56.05          | NaN          | No    |
| 3328 | 0      | 19.85          | NaN          | No    |
| 3823 | 0      | 25.35          | NaN          | No    |
| 4377 | 0      | 20.00          | NaN          | No    |
| 5215 | 0      | 19.70          | NaN          | No    |
| 6667 | 0      | 73.35          | NaN          | No    |
| 6751 | 0      | 61.90          | NaN          | No    |

Se eliminan los valores nulos utilizando la función `drop_na` y se eliminan columnas sin valor analítico para el problema de investigación.

```
df = df.dropna()
```

Eliminamos `customerID` ya que no sirve para el análisis

```
df = df.drop(columns=['customerID'])
```

Se procede a transformar la variable objetivo a 0s y 1s para el análisis, ya que no se pueden utilizar variables categóricas sin transformación en el modelo.

```
df['Churn'] = df['Churn'].map({'Yes': 1, 'No': 0})
```

## 7. EXPLORACIÓN NUMÉRICA

Para iniciar el análisis, se evalúan las estadísticas descriptivas de las variables numéricas, específicamente `Charges`, incluyendo la media, la desviación estándar y los cuartiles, utilizando la función `describe()`.

```
columnas = df[['MonthlyCharges', 'TotalCharges']]
estadisticas_descriptivas = columnas.describe()

print("Estadísticas descriptivas para las columnas:")
print(estadisticas_descriptivas)
```

Estadísticas descriptivas para las columnas:

|       | MonthlyCharges | TotalCharges |
|-------|----------------|--------------|
| count | 7032.000000    | 7032.000000  |
| mean  | 64.798208      | 2283.300441  |
| std   | 30.085974      | 2266.771362  |
| min   | 18.250000      | 18.800000    |
| 25%   | 35.587500      | 401.450000   |
| 50%   | 70.350000      | 1397.475000  |
| 75%   | 89.862500      | 3794.737500  |
| max   | 118.750000     | 8684.800000  |



Se procede a evaluar el porcentaje del problema de investigación, refiriéndose a la proporción de clientes que abandonan la empresa.

```
percent = df["Churn"].value_counts(normalize=True)
print(f"Porcentaje de clientes que se van: \n{percent}")
```

Porcentaje de clientes que se van:

| Churn |          |
|-------|----------|
| 0     | 0.734215 |
| 1     | 0.265785 |

Name: proportion, dtype: float64

Para analizar la fuga de clientes en función de otras variables e identificar los factores predominantes, se realizan cruces de variables que permiten observar qué porcentaje de clientes se va o se queda según cada categoría.

```
pd.crosstab(df["Contract"], df["Churn"], normalize="index")
```

| Contract       | Churn | 0        | 1        |
|----------------|-------|----------|----------|
| Month-to-month |       | 0.572903 | 0.427097 |
| One year       |       | 0.887228 | 0.112772 |
| Two year       |       | 0.971513 | 0.028487 |

```
pd.crosstab(df["PaymentMethod"], df["Churn"], normalize="index")
```

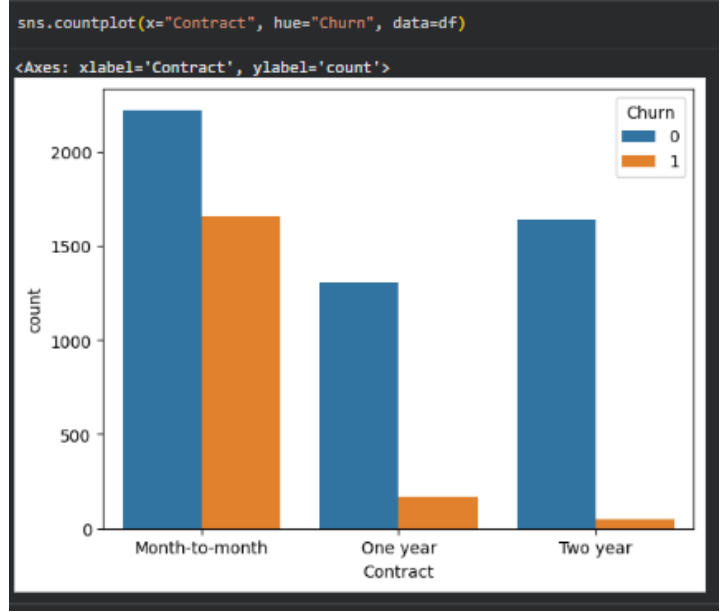
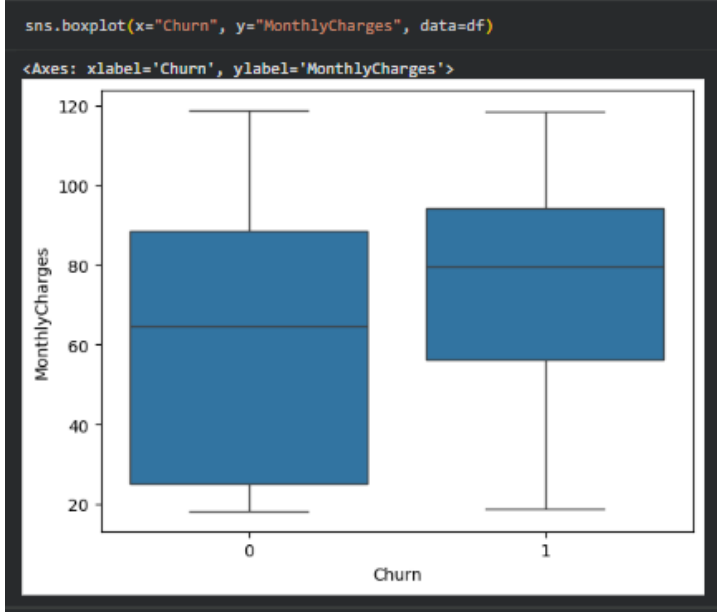
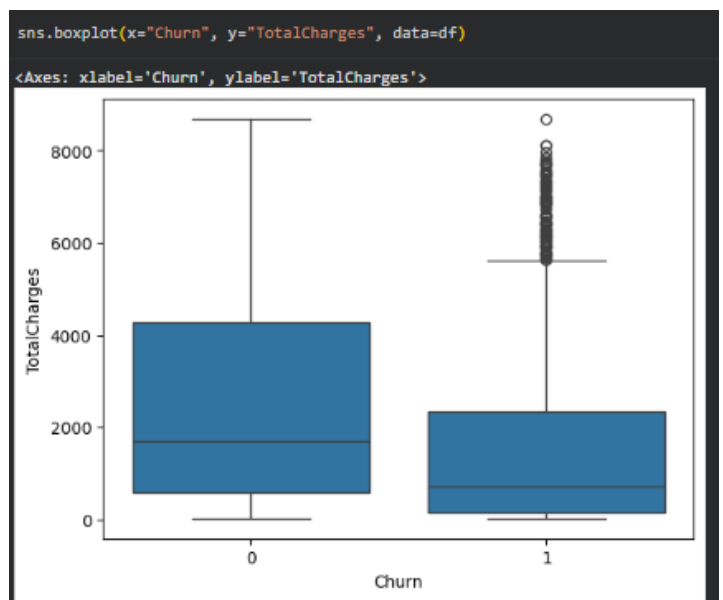
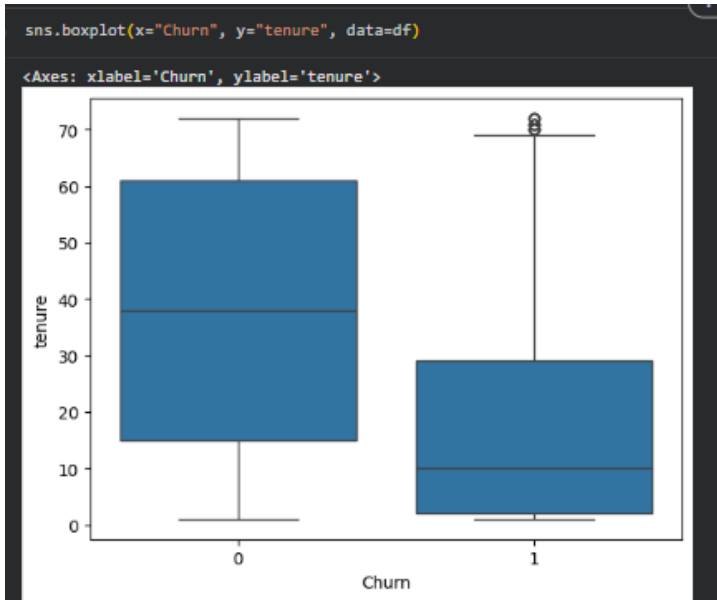
| PaymentMethod             | Churn | 0        | 1        |
|---------------------------|-------|----------|----------|
| Bank transfer (automatic) |       | 0.832685 | 0.167315 |
| Credit card (automatic)   |       | 0.847469 | 0.152531 |
| Electronic check          |       | 0.547146 | 0.452854 |
| Mailed check              |       | 0.807980 | 0.192020 |

```
pd.crosstab(df["InternetService"], df["Churn"], normalize="index")
```

| InternetService | Churn | 0        | 1        |
|-----------------|-------|----------|----------|
| DSL             |       | 0.810017 | 0.189983 |
| Fiber optic     |       | 0.581072 | 0.418928 |
| No              |       | 0.925658 | 0.074342 |

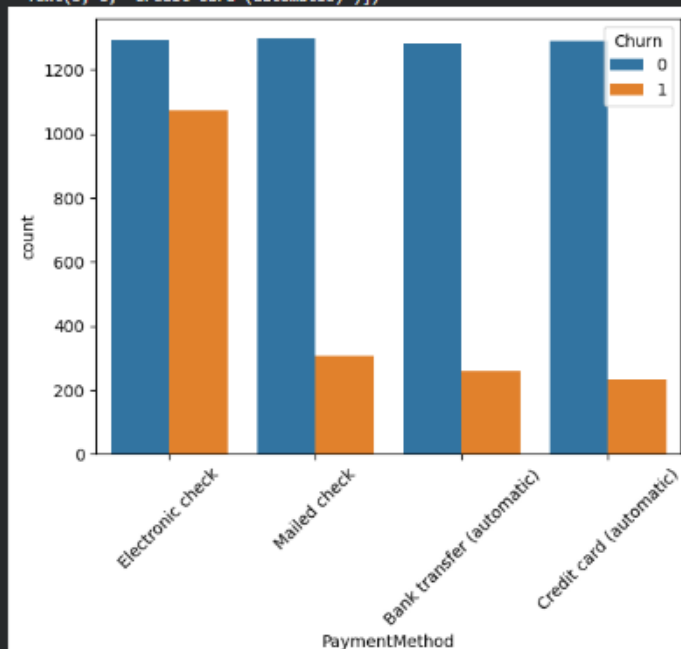
## 8. EXPLORACIÓN GRÁFICA

Para facilitar la comprensión, se utilizan gráficas que permiten visualizar los resultados obtenidos en la exploración numérica con gráficos de cajas y bigotes al igual que gráficos de barra utilizando la librería Seaborn a base de sus funciones `boxplot` y `countplot`.



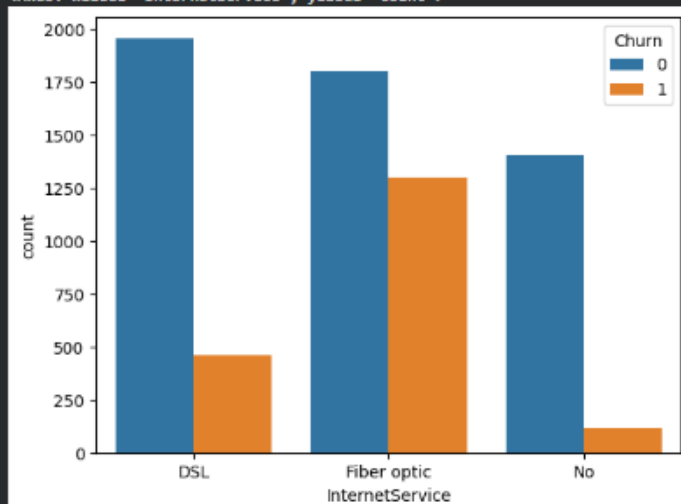
```
sns.countplot(x="PaymentMethod", hue="Churn", data=df)
plt.xticks(rotation=45)
```

```
[[0, 1, 2, 3],
 [Text(0, 0, 'Electronic check'),
  Text(1, 0, 'Mailed check'),
  Text(2, 0, 'Bank transfer (automatic)'),
  Text(3, 0, 'Credit card (automatic)')]]
```



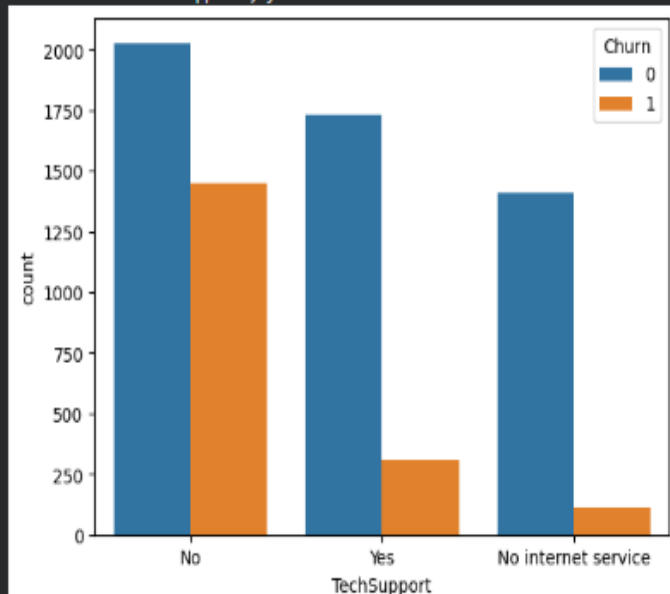
```
sns.countplot(x="InternetService", hue="Churn", data=df)
```

```
<Axes: xlabel='InternetService', ylabel='count'>
```



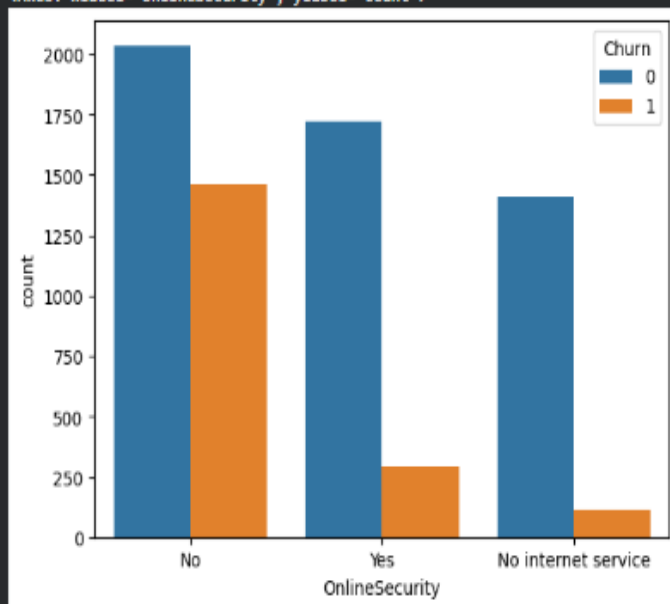
```
sns.countplot(x="TechSupport", hue="Churn", data=df)
```

```
<Axes: xlabel='TechSupport', ylabel='count'>
```



```
sns.countplot(x="OnlineSecurity", hue="Churn", data=df)
```

```
<Axes: xlabel='OnlineSecurity', ylabel='count'>
```



## 9. MODELADO

Para el modelado, se comienza con la transformación de variables categóricas a variables dummy, con el fin de que el modelo pueda utilizarlas en su análisis. Esta transformación consiste en convertir valores cualitativos en variables binarias, permitiendo que el modelo identifique y procese cada categoría de forma adecuada.

```
Creemos dummies para futuro modelado

categoricas = [
    'gender', 'Partner', 'Dependents',
    'PhoneService', 'MultipleLines', 'InternetService',
    'OnlineSecurity', 'OnlineBackup', 'DeviceProtection',
    'TechSupport', 'StreamingTV', 'StreamingMovies',
    'Contract', 'PaperlessBilling', 'PaymentMethod'
]

df_dummies = pd.get_dummies(df, columns=categoricas, dtype=int, drop_first=True)
df_powerbi = pd.get_dummies(df, columns=categoricas, dtype=int)

df_dummies.head(3)
```

|   | SeniorCitizen | tenure | MonthlyCharges | TotalCharges | Churn | gender_Male | Partner_Yes | Dependents_Yes | PhoneService_Yes | MultipleLines_No<br>phone service | ... | StreamingTV_No<br>internet<br>service | StreamingTV_Yes | Str |
|---|---------------|--------|----------------|--------------|-------|-------------|-------------|----------------|------------------|-----------------------------------|-----|---------------------------------------|-----------------|-----|
| 0 | 0             | 1      | 29.85          | 29.85        | 0     | 0           | 1           | 0              | 0                | 1                                 | ... | 0                                     | 0               |     |
| 1 | 0             | 34     | 56.95          | 1889.50      | 0     | 1           | 0           | 0              | 1                | 0                                 | ... | 0                                     | 0               |     |
| 2 | 0             | 2      | 53.85          | 108.15       | 1     | 1           | 0           | 0              | 1                | 0                                 | ... | 0                                     | 0               |     |

3 rows x 31 columns

Se construyó una matriz de correlación y se representó mediante un mapa de calor (*heatmap*), con el propósito de analizar de forma visual las relaciones existentes entre las variables dummies.



Luego, se dividen las columnas en variables independientes y variable dependiente para el modelado, utilizando aquellas que sean estadísticamente relevantes para el análisis.

```
caracteristicas_columnas = [  
    'SeniorCitizen', 'tenure', 'MonthlyCharges', 'TotalCharges',  
    'gender_Male', 'Partner_Yes', 'Dependents_Yes', 'PhoneService_Yes',  
    'MultipleLines_No phone service', 'MultipleLines_Yes',  
    'InternetService_Fiber optic', 'InternetService_No',  
    'OnlineSecurity_No internet service', 'OnlineSecurity_Yes',  
    'OnlineBackup_No internet service', 'OnlineBackup_Yes',  
    'DeviceProtection_No internet service', 'DeviceProtection_Yes',  
    'TechSupport_No internet service', 'TechSupport_Yes',  
    'StreamingTV_No internet service', 'StreamingTV_Yes',  
    'StreamingMovies_No internet service', 'StreamingMovies_Yes',  
    'Contract_One year', 'Contract_Two year', 'PaperlessBilling_Yes',  
    'PaymentMethod_Credit card (automatic)',  
    'PaymentMethod_Electronic check', 'PaymentMethod_Mailed check'  
]  
  
X = df_dummies[caracteristicas_columnas] #Independent variables  
y = df_dummies['Churn'] #Target variable
```

Para realizar el modelado, se implementa un método que construye y muestra los resultados de un modelo de regresión lineal, facilitando el análisis mediante la presentación de sus principales métricas y resultados estadísticos, incluyendo el R-squared, los valores p y t, así como los intervalos de confianza, entre otros.

```
def regressionResults(remove_cols=None):  
    X2 = X.copy()  
  
    #Deja usar una lista o un str  
    if remove_cols is not None:  
        if isinstance(remove_cols, str):  
            remove_cols = [remove_cols]  
        X2 = X2.drop(columns=remove_cols, errors='ignore')  
  
    #Entradas de la regresión  
    X2 = sm.add_constant(X2).rename(columns={'const': 'intercept'})  
    X2 = X2.apply(pd.to_numeric, errors='coerce')  
    y2 = pd.to_numeric(y, errors='coerce')  
  
    #Fit al modelo  
    est = sm.OLS(y2, X2)  
    est2 = est.fit()  
  
    #Intervalo de confianza  
    conf = est2.conf_int(alpha=0.05)  
  
    print(est2.summary())    # OLS = Ordinary Least Squares
```

Usando este mismo método, se realiza la eliminación de variables con valores p elevados ( $p > 0.05$ ), ya que representan una baja significancia estadística, y a base del resultado se evalúan factores que aumenten o reduzcan la fuga de clientes mediante la evaluación de sus t - values.

```
#Eliminación directa de variables con valores p muy elevados e irrelevantes
regressionResults(['MonthlyCharges', 'gender_Male', 'Partner_Yes', 'PaymentMethod_Credit card (automatic)', 'PaymentMethod_Mailed check', 'OnlineBackup_Yes', 'DeviceProtection_Yes'])
```

| OLS Regression Results |                  |                     |         |
|------------------------|------------------|---------------------|---------|
| Dep. Variable:         | Churn            | R-squared:          | 0.284   |
| Model:                 | OLS              | Adj. R-squared:     | 0.282   |
| Method:                | Least Squares    | F-statistic:        | 173.7   |
| Date:                  | Mon, 23 Mar 2026 | Prob (F-statistic): | 0.00    |
| Time:                  | 03:05:06         | Log-Likelihood:     | -3059.4 |
| No. Observations:      | 7032             | AIC:                | 6153.   |
| Df Residuals:          | 7015             | BIC:                | 6269.   |
| Df Model:              | 16               |                     |         |
| Covariance Type:       | nonrobust        |                     |         |

Luego, se identifican las variables X y Y para separarlas en el proceso de entrenamiento y prueba, eliminando previamente las variables estadísticamente no significativas identificadas anteriormente y dividiendo los datos en conjuntos de entrenamiento y prueba estableciendo un test size de 0.3, refiriéndose que el 70% de los datos será para entrenamiento y el 30% para evaluación del modelo.

```
X = X.drop(columns=['MonthlyCharges', 'gender_Male', 'Partner_Yes',
                    'PaymentMethod_Credit card (automatic)',
                    'PaymentMethod_Mailed check', 'OnlineBackup_Yes',
                    'DeviceProtection_Yes'], axis=1) #Independent variables

y = df['Churn'] #Target variable

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42
)
```

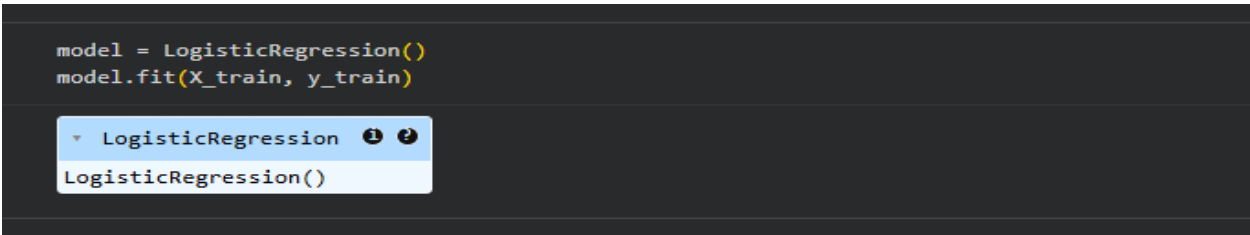
Se estandarizan las variables independientes por medio de MinMaxScaler para asegurar un análisis más efectivo, ya que, de esta forma todas las variables estarán en la misma escala y mejorara el desempeño del modelo.

```
scaler = MinMaxScaler()

X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)
```

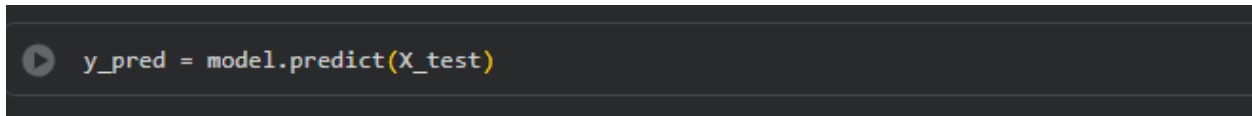
Luego de todo el proceso, se procede a entrenar el modelo de regresión logística utilizando las variables separadas en `X_train` y `y_train`, mediante la función `LogisticRegression()` y el método `model.fit()`.

```
model = LogisticRegression()
model.fit(X_train, y_train)
```



## 10. EVALUACIÓN DEL MODELO

Se utiliza el modelo entrenado para realizar predicciones con el fin de evaluar su desempeño y calcular su porcentaje de acierto, comparando `y_test` con `y_pred`.

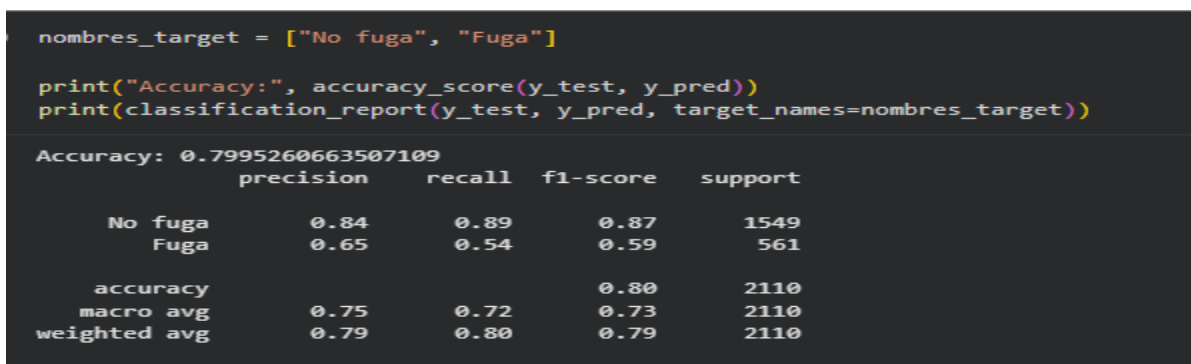


```
y_pred = model.predict(X_test)
```

Aquí se muestra el reporte de clasificación, donde se revisa la precisión, la sensibilidad (recall), el F1-score (promedio entre precisión y recall), y la cantidad real de casos de fuga y no fuga, ya que es esencial no solo basarse en la precisión, sino también en los valores reales, especialmente en la sensibilidad

```
nombres_target = ["No fuga", "Fuga"]

print("Accuracy:", accuracy_score(y_test, y_pred))
print(classification_report(y_test, y_pred, target_names=nombres_target))
```



|              | precision | recall | f1-score | support |
|--------------|-----------|--------|----------|---------|
| No fuga      | 0.84      | 0.89   | 0.87     | 1549    |
| Fuga         | 0.65      | 0.54   | 0.59     | 561     |
| accuracy     |           |        | 0.80     | 2110    |
| macro avg    | 0.75      | 0.72   | 0.73     | 2110    |
| weighted avg | 0.79      | 0.80   | 0.79     | 2110    |

Luego, para una evaluación más concisa del modelo, se calculan diferentes métricas con el fin de analizar su desempeño. Entre ellas se incluye el accuracy, que representa el total de predicciones correctas; la precisión, que mide qué tan acertadas fueron las predicciones positivas del modelo; y la sensibilidad, que refleja la cantidad de clientes que realmente se fueron y que el modelo logró identificar. También se evalúa la especificidad, que indica qué tan bien se identifican los clientes que no abandonaron la empresa y el F1-score, que representa el promedio entre la precisión y la sensibilidad. De esta manera, se obtiene una visión más completa de los resultados del modelo.

```
y_pred = y_pred.astype(int)

#Accuracy = (True positive + true negative)/Total predictions
#De todas las predicciones, cuántas fueron acertadas
accuracy_exactitud = accuracy_score(y_test, y_pred)
print("Accuracy: ", accuracy_exactitud)

#Precisión = True positive / (True positive + false positive)
#De todas las veces que el modelo dijo "se va", ¿cuántas veces se fue?
precision = metrics.precision_score(y_test, y_pred)
print("Precision: ", precision)

#Sensibilidad = True positive / (True positive + False negative)
#De todos los clientes que se iban, ¿Cuántos detectó el modelo?
sensibilidad_recall = metrics.recall_score(y_test, y_pred)
print("Sensibilidad: ", sensibilidad_recall)

#Especificidad = True negative / (True negative + false positive)
#De todos los clientes que se quedan, ¿Cuántos detectó el modelo?
especificidad = metrics.recall_score(y_test, y_pred, pos_label=0)
print("Especificidad: ", especificidad)

#f1 = 2 * ((precisión * sensibilidad) / (precision + sensibilidad))
f1_score = metrics.f1_score(y_test, y_pred)
print("F1 Score: ", f1_score)

print(f"\nDe todas las predicciones el {accuracy_exactitud*100:.2f}% fue acertado.")
print(f"De todas las veces que el modelo dijo \"se va\", el {precision*100:.2f}% fue acertado.")
print(f"De todos los clientes que se iban, el modelo detectó el {especificidad*100:.2f}%.")
print(f"De todos los clientes que se quedan, el modelo detectó el {accuracy_exactitud*100:.2f}%.")

Accuracy:  0.7995260663507109
Precision:  0.6487068965517241
Sensibilidad:  0.5365418894830659
Especificidad:  0.894770819883796
F1 Score:  0.5873170731707317
```



Aparte de las métricas, para evaluar el desempeño del modelo y observar cuántas predicciones fueron correctas y cuáles erróneas, se utiliza un apoyo visual mediante una matriz de confusión, la cual permite identificar en qué casos el modelo se equivocó y cómo fueron sus predicciones.

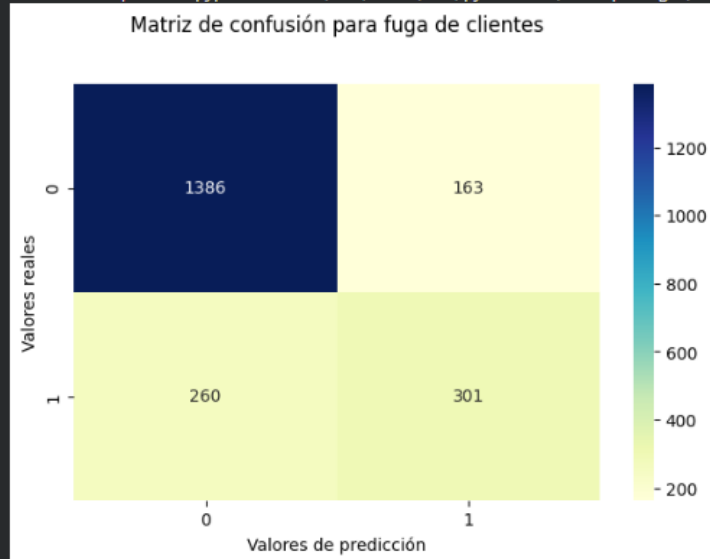
```
nombres_clases = ['No churn', 'Churn']
fig, ax = plt.subplots(figsize=(6,4))
marcas_verificacion = np.arange(len(nombres_clases))

plt.xticks(marcas_verificacion, nombres_clases)
plt.yticks(marcas_verificacion, nombres_clases)

sns.heatmap(pd.DataFrame(matriz_confusion), annot=True, fmt="g", cmap="YlGnBu")

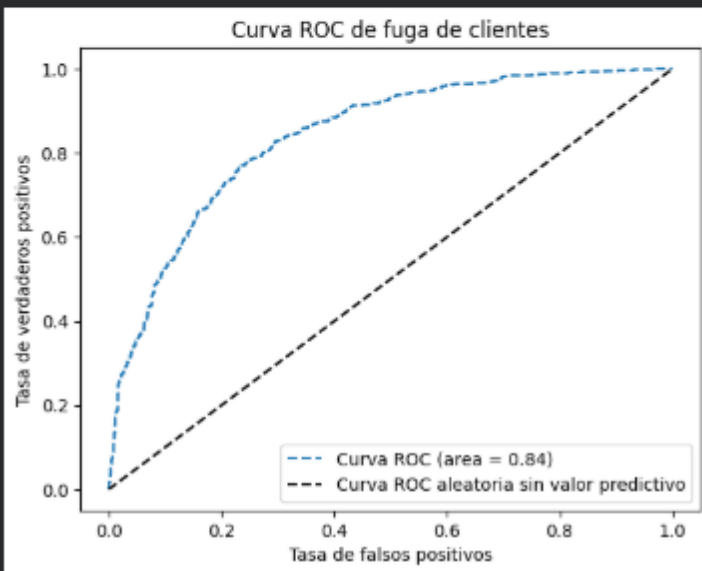
ax.xaxis.set_label_position("bottom")
plt.tight_layout()
plt.title("Matriz de confusión para fuga de clientes", y=1.1)
plt.ylabel("Valores reales")
plt.xlabel("Valores de predicción")
plt
```

<module 'matplotlib.pyplot' from '/usr/local/lib/python3.12/dist-packages/matplotlib/pyplot.py'>



Para culminar, en la evaluación del modelo se generó una curva ROC, la cual muestra la capacidad que tiene el modelo para distinguir entre los clientes que se van y los que se quedan.

```
ns_probs = [0 for _ in range(len(y_test))]  
y_test = y_test.astype(int)  
  
lr_probs = model.predict_proba(X_test)[:,1]  
ns_fpr, ns_tpr, thresholds = roc_curve(y_test, ns_probs)  
fpr, tpr, thresholds = roc_curve(y_test, lr_probs)  
  
auc = roc_auc_score(y_test, lr_probs)  
plt.plot(fpr, tpr, linestyle='--', label='Curva ROC (area = %0.2f)' % auc)  
plt.plot([0,1], [0,1], 'k--', label="Curva ROC aleatoria sin valor predictivo")  
plt.xlabel("Tasa de falsos positivos")  
plt.ylabel("Tasa de verdaderos positivos")  
plt.title('Curva ROC de fuga de clientes')  
plt.legend(loc=4)  
plt.show()
```



## 11. CONCLUSIONES

Luego de todo el proceso de análisis de datos, desde el tratamiento del conjunto de datos, la identificación de inconsistencias, su reparación y transformación, hasta la preparación del dataset para su uso en el modelo, y el uso de diversas librerías disponibles en Python, en este proyecto se logró desarrollar un modelo capaz de identificar y predecir casos de fuga y no fuga de clientes.

Según los resultados del análisis, se identificaron factores que incrementan la probabilidad de fuga, como no contar con tech support ni online security, utilizar métodos de pago como electronic check y tener contratos cortos como month-to-month. Por otro lado, los factores que favorecen la permanencia de los clientes incluyen contratos a largo plazo, así como contar con online security y tech support.

Basado en la evaluación del modelo, se obtuvo un accuracy cercano al 80%, lo que indica un desempeño bueno. Específicamente, el modelo logró identificar el 89.48% de los clientes que se van y el 79.95% de los clientes que se quedan. Sin embargo, se observa que el modelo tiene un mejor desempeño al predecir clientes que permanecen en la empresa que aquellos que la abandonan, ya que no logró identificar correctamente a **260 clientes** que sí se fueron, clasificándolos como clientes que permanecerían.

## 12. BIBLIOGRAFÍA

1. DataCamp. (s.f.). *Understanding logistic regression in Python*.  
<https://www.datacamp.com/tutorial/understanding-logistic-regression-python>
2. *classification\_report*. (s. f.). Scikit-learn. [https://scikit-learn.org/stable/modules/generated/sklearn.metrics.classification\\_report.html](https://scikit-learn.org/stable/modules/generated/sklearn.metrics.classification_report.html)
3. *Grouped boxplots — seaborn 0.13.2 documentation*. (s. f.).  
[https://seaborn.pydata.org/examples/grouped\\_boxplot.html](https://seaborn.pydata.org/examples/grouped_boxplot.html)
4. Vultr Docs. (2024, diciembre 27). *Python Pandas get\_dummies() - Convert to dummy variables*. [https://docs.vultr.com/python/third-party/pandas/get\\_dummies](https://docs.vultr.com/python/third-party/pandas/get_dummies)
5. *MinMaxScaler*. (s. f.). Scikit-learn. <https://scikit-learn.org/stable/modules/generated/sklearn.preprocessing.MinMaxScaler.html>